

Desafio JustTravel

Júlio César Moreira

¹ Salvador - Bahia

juliomoreira0111@gmail.com, juliomoreira@ufba.br

Aplicação 2ª fase Estágio Full Stack - Just Travel - Desafio Técnico

1. Introdução

O presente trabalho propõe o desenvolvimento de uma aplicação web *fullstack* para gerenciamento de tarefas, com funcionalidades como criação, exclusão, priorização, marcação de status e ordenação por data. A proposta visa demonstrar conhecimentos técnicos práticos em um sistema relativamente simples, porém completo, construído com tecnologias como Next.js (React.js), Flask e PostgreSQL.

A atividade contempla os seguintes objetivos:

- Projetar e implementar uma interface intuitiva com React.js e Tailwind CSS;
- Construir uma API RESTful utilizando Flask e SQLAlchemy;
- Persistir os dados em um banco PostgreSQL;
- Implementar funcionalidades como busca, filtros e ordenações;
- Realizar o *deploy* da aplicação em ambiente de produção utilizando Docker, Vercel e Render.

Tecnologias Utilizadas e Justificativa

Apesar de o desafio não especificar ferramentas de forma rígida, optou-se por utilizar tecnologias modernas que demonstram domínio de uma stack completa e compatível com ambientes de produção.

A escolha pelo **Next.js** com **TypeScript** no *frontend* se deu pelas melhorias que o framework traz em relação ao *create-react-app* tradicional, como o sistema de rotas integrado (mesmo que não utilizado diretamente), otimizações de performance e tipagem estática, o que aumenta a confiabilidade e a escalabilidade da interface.

No *backend*, embora tenha sido requisitado o uso do **Flask**, optou-se por integrar o **SQLAlchemy**, uma biblioteca ORM que facilita o mapeamento objeto-relacional e permite a escrita de consultas mais estruturadas.

Como banco de dados, foi utilizado o **PostgreSQL** em vez de alternativas mais simples, como SQLite ou persistência em arquivos JSON. Essa escolha se justifica pela necessidade de simular um ambiente de produção realista, conforme solicitado na descrição da vaga de estágio.

O uso do **Docker** visa facilitar a replicação do ambiente em diferentes máquinas, garantindo consistência entre desenvolvimento e produção. As plataformas **Render** (para o backend) e **Vercel** (para o frontend) foram escolhidas para o *deploy* por oferecerem integração direta com repositórios Git e serem compatíveis com as tecnologias adotadas, sobretudo o Docker para o backend.

2. Backend

Na atividade, foi solicitado o uso da linguagem Python e o framework Flask no desenvolvimento do *backend*. Enquanto o SQLAlchemy foi escolhido por como ORM (Object-Relational Mapping) devido sua documentação e popularidade. A estrutura do *backend* está organizada da seguinte forma:

- **orm/** – contém os modelos e definições de tabelas via `schema.py`;
- **src/** – agrupa os *controllers* e *routes* que organizam a lógica de negócio e as rotas da API;
- **.env** – arquivo de variáveis de ambiente que armazena configurações como URL do banco de dados e porta da aplicação;
- **app.py** – arquivo principal que inicializa a aplicação Flask;
- **requirements.txt** – lista de dependências necessárias para a aplicação;
- **backend.dockerfile** – arquivo de configuração Docker para empacotar o serviço;
- **compose.yml** – define os serviços e volumes do ambiente Docker;
- **seeds.py** – script para inserção de dados iniciais no banco.

2.1. Frontend

A estrutura do *frontend* está contida na pasta `src`, que por sua vez possui os seguintes diretórios e arquivos principais:

- **app/** – contém o arquivo `page.tsx`, responsável por renderizar a interface principal da aplicação. Nesse arquivo estão centralizados os componentes visuais e também as funções de `fetch` que consomem a API do *backend*;
- **components/** – abriga os componentes reutilizáveis da aplicação:
 - `Header.tsx` – componente responsável pelo cabeçalho da aplicação;
 - `Footer.tsx` – componente que representa o rodapé da aplicação;
 - `Card.tsx` – componente que exibe os cards de tarefas criadas pelo usuário;
 - `Popover.tsx` – componente em formato de *pequena janela*, utilizado para criar novas tarefas;
 - `Sheet.tsx` – componente que exibe uma tela lateral com as tarefas já concluídas.
- **components/ui/** – diretório que contém componentes de suporte, como `Popover.tsx` e `Sheet.tsx`, customizados a partir de bibliotecas como ShadCN UI;
- **.env.development** e **.env.production** – arquivos utilizados para configurar variáveis de ambiente específicas para os ambientes de desenvolvimento e produção, como a URL da API;
- **frontend.dockerfile** – arquivo responsável pela configuração da imagem Docker do *frontend*.

3. Explicação e exemplificação do código Backend

3.1. Controlador de Tarefas

A classe `TaskController` centraliza a lógica de controle para as operações relacionadas ao gerenciamento de tarefas. Ela implementa os métodos responsáveis por tratar requisições HTTP e interagir com os modelos definidos no ORM.

Cada função será explicada individualmente a seguir, com sua respectiva rota e comportamento esperado.

1. `get_all_tasks()`

Objetivo: Retorna todas as tarefas cadastradas no sistema, ordenadas por ID.

Funcionamento: Consulta a tabela de tarefas utilizando o método `.query.order_by().all()` e converte cada tarefa em um dicionário com os seguintes campos: `id`, `titulo`, `descricao`, `status`, `prioridade`, `criado_em` e `atualizado_em`.

Exemplo de rota:

GET `/tasks`

Resposta esperada:

```
[
  {
    "id": 1,
    "titulo": "Estudar Flask",
    "descricao": "Ler a documentação oficial",
    "status": false,
    "prioridade": "media",
    "criado_em": "2024-07-10",
    "atualizado_em": "2024-07-10"
  }
]
```

2. `create_task()`

Objetivo: Criar uma nova tarefa com base nos dados recebidos via JSON.

Funcionamento: Valida a presença do título, atribui valores padrão para os demais campos (como prioridade média e status falso) e insere a nova tarefa no banco de dados com `db.session.add()`.

Exemplo de rota:

POST `/tasks`

Content-Type: `application/json`

Corpo da requisição:

```
{
  "titulo": "Finalizar relatório",
  "descricao": "Adicionar exemplo das rotas",
  "prioridade": "alta",
  "status": false
}
```

Resposta esperada:

```
{
  "id": 2,
  "titulo": "Finalizar relatório",
  "descricao": "Adicionar exemplo das rotas",
  "prioridade": "alta",
  "status": false,
  "criado_em": "2024-07-14T22:00:00",
  "atualizado_em": "2024-07-14T22:00:00"
}
```

3. update_task(task_id)

Objetivo: Alternar o status da tarefa (de true para false ou vice-versa), com base no ID fornecido.

Funcionamento: Busca a tarefa pelo ID, inverte o campo status e atualiza a data de modificação. Em seguida, salva a alteração no banco.

Exemplo de rota:

PUT /tasks/2

Resposta esperada:

```
{
  "id": 2,
  "titulo": "Finalizar relatório",
  "descricao": "Adicionar exemplo das rotas",
  "prioridade": "alta",
  "status": true,
  "criado_em": "2024-07-14T22:00:00",
  "atualizado_em": "2024-07-14T22:30:00"
}
```

4. delete_task(task_id)

Objetivo: Remover uma tarefa do sistema com base no seu ID.

Funcionamento: A função busca a tarefa pelo ID e, se encontrada, a deleta com `db.session.delete()`. Retorna os dados da tarefa excluída.

Exemplo de rota:

DELETE /tasks/2

Resposta esperada:

```
{
  "mensagem": "Tarefa deletada com sucesso",
  "tarefa": {
    "id": 2,
```

```

        "titulo": "Finalizar relatório",
        "descricao": "Adicionar exemplo das rotas",
        "prioridade": "alta",
        "status": true,
        "criado_em": "2024-07-14T22:00:00",
        "atualizado_em": "2024-07-14T22:30:00"
    }
}

```

3.2. Definição de Rotas e Inicialização da Aplicação

A definição das rotas da API ocorre no arquivo `routes.py`, utilizando o recurso de *Blueprints* do Flask, que permite modularizar a aplicação. Já a inicialização da aplicação Flask e a configuração do banco de dados ocorrem no arquivo `app.py`.

1. routes.py

As rotas da API são agrupadas sob o prefixo `/tasks`. Cada rota está associada a um método específico da classe `TaskController`:

- GET `/tasks` – Retorna todas as tarefas cadastradas;
- POST `/tasks` – Cria uma nova tarefa;
- PUT `/tasks/<id>` – Atualiza o status de uma tarefa (alterna entre concluída e pendente);
- DELETE `/tasks/<id>` – Remove a tarefa do banco de dados.

As rotas são registradas via o objeto `Blueprint`, conforme o seguinte trecho de código:

```

task_bp = Blueprint('task_bp', __name__)

task = TaskController()
task_bp.route('/', methods=['GET'])(task.get_all_tasks)
task_bp.route('/', methods=['POST'])(task.create_task)
task_bp.route('/<int:task_id>', methods=['PUT'])(task.update_task)
task_bp.route('/<int:task_id>', methods=['DELETE'])(task.delete_task)

```

2. app.py

O arquivo `app.py` representa o ponto de entrada da aplicação. Nele são configuradas:

- A URL de conexão com o banco de dados (`DATABASE_URL`), via variáveis de ambiente;
- A extensão `SQLAlchemy` para ORM;
- O uso do `CORS` para permitir requisições de origens diferentes (útil para integração com o frontend);
- O registro das rotas via `Blueprint`;
- Um endpoint auxiliar em `/` que responde com uma mensagem simples.

Além disso, o banco é inicializado com `db.create_all()` quando a aplicação roda em modo local.

```
app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = os.getenv("DATABASE_URL")
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db.init_app(app)
CORS(app, resources={r"/*": {"origins": "*"}})

app.register_blueprint(task_bp, url_prefix='/tasks')

@app.route('/')
def hello():
    return 'Hello, _Flask!'

if __name__ == '__main__':
    with app.app_context():
        db.create_all()
    app.run(host='0.0.0.0', port=3001, debug=True)
```

Essa estrutura modular facilita a manutenção, reutilização e escalabilidade do código no futuro.

4. Explicação e exemplificação do código Frontend

A interface do usuário da aplicação foi desenvolvida com **Next.js** e **React**, utilizando também a biblioteca **Tailwind CSS** para estilização. A seguir, é descrita a lógica principal da página inicial, que concentra o gerenciamento das tarefas.

4.1. Página Principal: `page.tsx`

A página inicial (`page.tsx`) é responsável por renderizar, filtrar, ordenar e manipular as tarefas. As funções principais são descritas a seguir:

1. Estados globais

- `cards` – lista de tarefas recebidas da API.
- `searchTerm` – termo de busca digitado pelo usuário.
- `sortType` – define se a ordenação será por **data** de criação ou por **urgência**.

2. `createTask`

Função responsável por enviar uma nova tarefa para a API. Ela formata a data de criação e atualização antes de enviar o conteúdo via método POST.

```
await fetch(`${API_URL}/tasks/`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(taskWithDates)
})
```

3. fetchTasks

Busca todas as tarefas cadastradas na API e atualiza o estado cards. Essa função é chamada uma única vez no carregamento da página via useEffect.

4. removeTask

Responsável por excluir uma tarefa com base no seu ID. Após o sucesso da requisição DELETE, remove localmente a tarefa da lista exibida.

5. toggleTaskStatus

Altera o status de uma tarefa entre true e false, usando a rota PUT. Também atualiza o estado global cards com a nova informação.

6. Filtros e Ordenações

As tarefas pendentes são filtradas com base no termo de busca presente no título. A seguir, são ordenadas:

- Por data de criação (mais antiga primeiro);
- Ou por prioridade (da mais urgente para a menos urgente).

```
// Ordena por data
if (sortType === 'data') {
  sortedPendingTasks.sort((a, b) =>
    new Date(a.criado_em).getTime() - new Date(b.criado_em).getTime()
  )
}
// Ordena por urgência
else {
  const priorityOrder = ['baixa', 'media', 'alta', 'urgente']
  sortedPendingTasks.sort((a, b) =>
    priorityOrder.indexOf(b.prioridade) - priorityOrder.indexOf(a.prioridade)
  )
}
```

7. Renderização dos Componentes

A página é dividida nos seguintes componentes:

- Header – barra de topo com campos de filtro;
- Popover – formulário flutuante para criação de tarefas;
- Sheet – painel lateral com tarefas concluídas;
- Card – componente visual que representa cada tarefa individual.

A lógica da aplicação mantém a interface reativa e interativa, refletindo as operações feitas pelo usuário (criar, excluir, filtrar, alternar status etc.) de forma imediata na interface.

4.2. Componente `Header.tsx`

O componente Header representa a barra superior da aplicação. Ele fornece ao usuário duas funcionalidades principais: a busca por título de tarefas e a ordenação por critério de exibição.

1. Props

O componente recebe duas funções como propriedades, permitindo que o controle de estado seja mantido no componente pai:

- `onFilterChange(term)` – função disparada sempre que o valor do campo de busca for alterado;
- `onFilterTypeChange(type)` – função disparada ao escolher o tipo de ordenação (*data* ou *urgência*).

2. Estado Local

Internamente, o componente mantém dois estados:

- `searchTerm` – representa o conteúdo atual digitado no campo de busca;
- `filterType` – armazena o tipo de ordenação selecionada.

3. Comportamento

Ao digitar no campo de busca, o estado `searchTerm` é atualizado e a função `onFilterChange` é chamada. Da mesma forma, ao clicar em um dos botões de ordenação, o estado `filterType` é alterado e a função `onFilterTypeChange` é executada.

```
// Exemplo da função chamada ao alterar a busca
const handleSearchInputChange = (e) => {
  const value = e.target.value
  setSearchTerm(value)
  onFilterChange(value)
}
```


4. Interface Visual

O componente apresenta:

- Um título fixo à esquerda (“Home”);
- Um campo de entrada centralizado para busca de tarefas;
- Dois botões de ordenação: por “Data” ou “Urgência”, com destaque visual baseado no estado selecionado.

O layout foi construído com Tailwind CSS, garantindo responsividade e destaque visual, conforme mostrado no exemplo abaixo:

```
<input
  type="text"
  placeholder="Pesquisar_Tasks..."
  className="w-full_px-4_py-2_rounded-md_border"
/>

<button className="bg-blue-500_text-white">Data</button>
<button className="bg-white_text-blue-500">Urgência</button>
```

Esse componente melhora significativamente a usabilidade da aplicação ao permitir controle dinâmico da visualização das tarefas com base nos critérios mais relevantes para o usuário.

4.3. Componente Popover.tsx

O componente Popover é um formulário flutuante construído com o padrão de *popover*, utilizado para criar novas tarefas. Ele é acessado via um botão destacado na interface principal e usa elementos visuais da biblioteca **ShadCN UI**.

1. Props

- `onPost(data)` – função passada pelo componente pai responsável por enviar os dados da nova tarefa à API.

2. Estado Local

- `title` – título da tarefa;
- `description` – descrição da tarefa;
- `priority` – prioridade selecionada pelo usuário.

Esses valores são atualizados conforme o usuário interage com os campos de entrada.

3. Comportamento do Formulário

Ao preencher os campos e clicar no botão **Salvar Task**, o método `handleSubmit` é chamado. Essa função monta o objeto da tarefa e o envia para a função `onPost`, limpando os campos após o envio.

```
const handleSubmit = () => {
  onPost({
    titulo: title,
    descricao: description,
    prioridade: priority,
    status: false
  });
  setTitle('');
  setDescription('');
  setPriority('media');
}
```

O botão só pode ser acionado quando o campo de título não está vazio, evitando o envio de tarefas inválidas.

4. Interface Visual

O Popover abre uma janela flutuante com:

- Campo de texto para o título;
- Área de texto para descrição;
- Seleção de prioridade (baixa, média, alta, urgente);
- Botão para salvar.

Exemplo de código HTML com Tailwind CSS:

```
<select class="rounded border-2 shadow-sm">
  <option value="baixa">Baixa</option>
  <option value="media">Média</option>
  <option value="alta">Alta</option>
  <option value="urgente">Urgente</option>
</select>
```

O botão de ativação do formulário é destacado com a cor azul e texto branco, com efeito *hover*, garantindo que o usuário o perceba facilmente na interface:

```
<button class="bg-blue-500 text-white hover:bg-blue-700">
  Criar Task
</button>
```

5. Usabilidade

Esse componente garante que a criação de tarefas seja feita de maneira rápida e fluida, sem a necessidade de redirecionamento ou recarregamento de página, oferecendo uma boa experiência ao usuário.

4.4. Componente CompletedTasksSheet.tsx

O componente Sheet permite a visualização das tarefas que foram marcadas como concluídas. Ele é exibido em formato de *tela lateral (sheet)* e utiliza a biblioteca **ShadCN UI**.

1. Props

Esse componente recebe três propriedades principais:

- `completedTasks` – um array de tarefas cuja propriedade `status` está marcada como verdadeira;
- `deleteCard(id)` – função responsável por excluir uma tarefa com base no seu identificador;
- `toggleCardStatus(id, status)` – função para alternar o status de uma tarefa (de completa para pendente).

2. Comportamento

O botão “Tasks Completas” abre uma janela lateral (Sheet) contendo uma grade responsiva de Cards renderizados com os dados das tarefas concluídas. Cada card inclui as opções de:

- reverter o status (torná-la pendente novamente);
- excluir permanentemente a tarefa.

```
// Exemplo de iteração sobre tarefas concluídas
{completedTasks.map((task) => (
  <Card
    key={task.id}
    titulo={task.titulo}
    descricao={task.descricao}
    status={task.status}
    prioridade={task.prioridade}
    criado_em={task.criado_em}
    atualizado_em={task.atualizado_em}
    deleteCardProps={deleteCard}
    toggleCardStatus={() => toggleCardStatus(task.id, task.status)}
  />
)}}}
```

3. Interface Visual

A interface segue um padrão minimalista e adaptável:

- A Sheet ocupa o lado direito da tela, com rolagem vertical habilitada;
- Os cards são dispostos em grade flexível para dispositivos maiores;
- Um botão de destaque inicia a visualização da lista de tarefas concluídas.

```
<button class="bg-blue-500_hover:bg-blue-700_text-white_font-bold">
  Tasks Completas
</button>
```

4. Responsividade

A estrutura da grade (`flex-wrap`) garante que os cards sejam organizados corretamente em telas pequenas e grandes, mantendo boa experiência de navegação.

5. Usabilidade

Esse componente melhora a experiência do usuário ao separar as tarefas concluídas das pendentes, mantendo a tela principal mais limpa. A escolha de um painel lateral retrátil evita o uso de páginas adicionais e mantém o foco na facilidade.

4.5. Componente `Card.tsx`

O componente `Card` é o elemento visual central da aplicação, responsável por representar uma tarefa individual, exibindo seus detalhes, status e ações possíveis (como excluir ou finalizar).

1. Props Recebidas

O componente recebe as seguintes propriedades:

- `index`: identificador da tarefa;
- `titulo`: título da tarefa;
- `descricao`: descrição da tarefa;
- `status`: booleano que indica se a tarefa foi concluída;
- `prioridade`: uma das opções: baixa, media, alta, urgente;
- `criado_em`: data de criação da tarefa;
- `atualizado_em`: data da última atualização da tarefa;
- `deleteCardProps(index)`: função para remover a tarefa;
- `toggleCardStatus(index, status)`: função para alternar o status da tarefa.

2. Funcionalidade

O `Card` apresenta as seguintes funções principais:

- Exibir os dados principais da tarefa (título, prioridade, descrição e datas);
- Mostrar o status atual (pendente ou finalizada);
- Permitir alternar entre visualizar mais/menos da descrição;
- Permitir exclusão da tarefa;
- Alternar entre tarefa concluída e pendente.

3. Comportamento Condicional

- Caso a descrição ultrapasse 5 linhas, um botão “Mostrar mais / Mostrar menos” é exibido automaticamente;
- Caso a tarefa esteja expandida, exibe todo o conteúdo e botão para recolher;

- Caso esteja colapsada, mostra apenas informações básicas e o botão de “Finalizar” ou “Reativar”.

```
// Exemplo de alternância de status da tarefa
<button onClick={() => toggleCardStatus(index, status)}>
  {!status ? 'Finalizar Task' : 'Reativar Task'}
</button>
```

4. Cores e Destaques Visuais

A borda lateral do Card muda de cor com base na prioridade da tarefa, conforme o seguinte mapeamento:

- baixa: verde claro;
- media: amarelo;
- alta: laranja;
- urgente: vermelho.

Essa distinção visual contribui para a identificação rápida de tarefas mais importantes.

```
// Mapeamento de cores por prioridade
const priorityClassMap = {
  baixa: 'bg-green-100 border-green-400',
  media: 'bg-yellow-100 border-yellow-400',
  alta: 'bg-orange-100 border-orange-400',
  urgente: 'bg-red-100 border-red-400',
}
```

5. Acessibilidade e Usabilidade

- Ícone de “X” para deletar a tarefa localizado no canto superior direito;
- Botões de ação visíveis apenas quando o card não está expandido;
- Componente responsivo, com largura máxima de 350px.

5. Docker e docker-compose

A aplicação é containerizada utilizando **Docker** e orquestrada com **Docker Compose**, o que permite fácil replicação, escalabilidade e isolamento dos serviços.

5.1. Dockerfile do Backend

O arquivo `backend.dockerfile` define o ambiente de execução do servidor backend (Flask). Ele realiza os seguintes passos:

Algoritmo 1. Dockerfile do Backend

```
FROM python:3.11-slim
```

```
WORKDIR /backend
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
RUN chmod +x entrypoint.sh
```

```
EXPOSE 3001
```

```
ENTRYPOINT ["/entrypoint.sh"]
```

- Utiliza uma imagem leve do Python 3.11;
- Define o diretório de trabalho como /backend;
- Instala as dependências listadas em requirements.txt;
- Copia o código-fonte para dentro do container;
- Define permissão de execução para o entrypoint.sh, que inicia o servidor;
- Expõe a porta 3001, utilizada pela aplicação Flask.

5.2. Arquivo docker-compose.yml

O docker-compose.yml define os quatro serviços da aplicação:

Algoritmo 2. docker-compose.yml

```
version: '3.8'
```

```
services:
```

```
  backend:
```

```
    build:
```

```
      context: .
```

```
      dockerfile: backend.dockerfile
```

```
    env_file:
```

```
      - .env
```

```
    ports:
```

```
      - "3001:3001"
```

```
    environment:
```

```
      - DATABASE_URL=postgresql://postgres:postgres@db:5432/postgres
```

```
    depends_on:
```

```
      - db
```

```
  db:
```

```
    container_name: db
```

```
    image: postgres:12
```

```
    environment:
```

```

    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: postgres
  ports:
    - "5432:5432"
  volumes:
    - pgdata:/var/lib/postgresql/data

pgadmin:
  container_name: pgadmin
  image: dpage/pgadmin4:latest
  restart: always
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@admin.com
    PGADMIN_DEFAULT_PASSWORD: admin
  ports:
    - "8080:80"
  depends_on:
    - db
  volumes:
    - /etc/localtime:/etc/localtime:ro
    - /etc/timezone:/etc/timezone:ro

frontend:
  build:
    context: ../front-end
    dockerfile: frontend.dockerfile
  env_file:
    - ../front-end/.env.development
  ports:
    - "3000:3000"
  command: ["npm", "run", "dev"]
  depends_on:
    - backend

volumes:
  pgdata:

```

Resumo dos Serviços

- **backend**: servidor Flask que responde pela API REST (/tasks), construído com base no Dockerfile;
- **db**: banco de dados PostgreSQL persistente, com volume nomeado pgdata;
- **pgadmin**: interface web para administração do banco de dados;
- **frontend**: aplicação em React com Next.js que consome a API do backend.

Volumes

O volume `pgdata` é usado para manter os dados do PostgreSQL persistentes mesmo que o container seja destruído.

Relações de Dependência

- O backend depende do banco de dados (db);
- O frontend depende do backend estar pronto;
- O pgAdmin depende do banco de dados para funcionar.

5.3. Dockerfile do Frontend

O Dockerfile do frontend define o ambiente para executar a aplicação web construída com **Next.js** e **React**. Ele realiza os seguintes passos:

Algoritmo 3. Dockerfile do Frontend

```
FROM node:20

WORKDIR /front-end

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "run", "dev"]
```

- Utiliza a imagem oficial do Node.js na versão 20;
- Define o diretório de trabalho como `/front-end`;
- Copia os arquivos `package.json` e `package-lock.json` (se existir);
- Executa `npm install` para instalar as dependências do projeto;
- Copia o restante do código-fonte da aplicação;
- Expõe a porta 3000, usada pelo servidor de desenvolvimento do Next.js;
- Define o comando padrão para iniciar o projeto: `npm run dev`.

Observação: esse Dockerfile está configurado para desenvolvimento local, utilizando o modo *hot-reload* do Next.js. Em ambientes de produção, o ideal seria realizar um build estático da aplicação e utilizar um servidor mais enxuto (como `nginx`) para servir os arquivos.

6. Aprendizados

Durante o desenvolvimento do projeto, foram consolidados diversos conhecimentos técnicos, entre os quais se destacam:

- Estruturação de APIs REST com Flask e SQLAlchemy;
- Manipulação de estados e componentes com React e TypeScript;
- Comunicação assíncrona com o backend por meio de requisições HTTP;
- Uso de containers com Docker para orquestração e padronização do ambiente de desenvolvimento;
- Integração entre múltiplos serviços com Docker Compose;
- Estilização com TailwindCSS e foco em design responsivo;
- Escrita de código limpo e reutilizável com organização por componentes;
- Versionamento e deploy com práticas modernas de DevOps.

Além dos conhecimentos técnicos, também foram desenvolvidas habilidades interpessoais como autonomia, organização e tomada de decisões técnicas, que são essenciais na atuação profissional da área de computação. ências (gerada automaticamente)